

Welcome to this masterclass on architecting and engineering a production-grade **Prometheus Monitoring and Observability Stack**. As a DevOps trainer, I will guide you through moving away from standard "hello-world" Docker setups and focus on building a resilient, secure, and highly scalable metrics infrastructure designed for real-world enterprise workloads.  Medium +1

Here is a comprehensive deployment and configuration manual for a modern enterprise monitoring architecture.

Production-Grade Prometheus Implementation & Observability Guide

1. Production Architecture Philosophy

In enterprise environments, monitoring is a **Tier-0 Service** that must remain operational during application failures. Prometheus uses a **Pull-Based Model** which provides a decoupled architecture, built-in protection against target overloads through controlled scraping intervals, and dynamic service discovery for ephemeral workloads.

2. Production-Grade Installation & Configuration

For production deployments on isolated VMs, avoid raw binaries and use a hardened, **Systemd**-managed setup with dedicated service accounts and explicit storage boundaries.

- **OS Preparation:** Create unprivileged users (`prometheus` , `node_exporter`) and provision restricted directories `/etc/prometheus` and `/var/lib/prometheus` .
- **Binary Deployment:** Download locked-down stable binaries (e.g., v2.54.0), move executables to `/usr/local/bin` , and configure asset paths.
- **Engine Configuration (`prometheus.yml`):** Define global scrape and evaluation intervals (typically 15s), add external context labels (`environment` , `region`), and specify scrape jobs for Prometheus itself and infrastructure targets.
- **Systemd Service Unit:** Run Prometheus with flags for retention limits (e.g., `--storage.tsdb.retention.time=30d` and `--storage.tsdb.retention.size=50GB`), admin APIs, and lifecycle management enabled.

3. The 4 Core Metric Types

Prometheus processes data across four primary metric structures:

- **Counter:** Monotonically increasing counters (e.g., total HTTP requests).
- **Gauge:** Values that go up and down arbitrarily (e.g., memory utilization).
- **Histogram:** Samples observations into configurable buckets with sums and counts.
- **Summary:** Client-side calculations of configurable sliding-window quantiles.

4. Production Exporters: Systemd Infrastructure Setup

Exporters convert system and application telemetry into scrape-ready HTTP

`/metrics` endpoints. **Node Exporter** handles hardware and OS metrics.

Production deployments should disable noisy, unused collectors (like wifi or zfs) via Systemd flags to conserve host resources.

5. PromQL Engine Cheat Sheet

PromQL evaluates time-series metrics for real-time alerting and capacity planning:

- **CPU Utilization:** Calculates multi-core idle percentages over a sliding window.
- **Memory Exhaustion:** Computes active vs. total system memory.
- **p99 Latency:** Evaluates sub-second API latency distribution using `histogram_quantile`.
- **Error Rate:** Tracks real-time ratios of 5xx HTTP responses.

6. Enterprise Scaling Strategy

As metrics volume grows, standalone instances face I/O bottlenecks. Scale securely using:

- **Long-Term Storage:** Integrate distributed engines like Thanos, Cortex, or Mimir via `remote_write`.
- **High Availability:** Run mirrored, redundant Prometheus scrapers feeding a deduplicated Alertmanager cluster.

To help tailor this demonstration to your specific environment, let me know:

- What is your **target deployment platform**? (e.g., Bare Linux VMs, Docker Compose, or a Kubernetes cluster using the `kube-prometheus-stack` Helm chart?)
- What **kinds of applications** do you need to monitor? (e.g., Java/Spring Boot, Go microservices, Python APIs, or third-party legacy databases like MySQL/PostgreSQL?)
- Are there specific **Service Level Objectives (SLOs)** or alerting integrations (like Slack, PagerDuty, or Email) you want to include in this training plan?  AWS in Plain English +4



 Medium

Production Monitoring Made Easy: Prometheus, Grafana, and ...

Bootcamp * From Zero to Observability: Building a Production-Grade Monitoring Stack with Prometheus & Grafana * Introduction * By ...



 Medium

Prometheus Monitoring: Complete Setup & Best Practices (Beginner ...

Prometheus Monitoring: Complete Setup & Best Practices (Beginner ... * What is Prometheus? * Key Features * Architecture Overview ...



 Networkers Home

Monitoring with Prometheus & Grafana - Networkers Home

Prometheus's powerful query language, PromQL, enables complex data analysis and alerting conditions. It supports aggregation, filt...

Show all

explain step by step how i can implement prometheus monitoring solution using docker compose. Add all the ...



Here is your file:

Prometheus-Production-Stack

 ZIP

This package contains a complete, deployment-ready blueprint containing structural manifests configured for a production-hardened Docker Compose monitoring topology.

Production Topology Architecture Overview

A production-grade environment differs from standard configurations by explicitly containing storage limits, enforcing network isolation, implementing proactive container health checks, restricting host expose vectors (`127.0.0.1`), and optimizing kernel collection hooks.

```
.
├── docker-compose.yml
└── prometheus/
    ├── alert.rules.yml
    └── prometheus.yml
```

Step-by-Step Implementation Framework

Step 1: Provisioning the Directory Architecture

Execute these terminal commands on your target Linux machine to set up the workspace and establish configuration tracking.

```
bash
```

```
mkdir -p prometheus-stack/prometheus
cd prometheus-stack
```

Use code with caution.

Step 2: Configure the Service Orchestra (`docker-compose.yml`)

Create the orchestrator configuration file inside the root directory. This config locks production versions, mounts standard Linux system boundaries (`/proc` , `/sys`) safely, defines rotation-capped log arrays, and ties container execution onto a single isolated software bridge (`monitoring-net`).

yaml

version: '3.8'

services:

prometheus:

image: prom/prometheus:v2.54.0

container_name: prometheus-prod

restart: unless-stopped

volumes:

- ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
- ./prometheus/alert.rules.yml:/etc/prometheus/alert.rules.yml:r
- prometheus-data:/prometheus

command:

- '--config.file=/etc/prometheus/prometheus.yml'
- '--storage.tsdb.path=/prometheus'
- # Production Hardening: Explicitly cap retention to guard disk L*
- '--storage.tsdb.retention.time=30d'
- '--storage.tsdb.retention.size=50GB'
- '--web.enable-lifecycle'
- '--web.console.templates=/usr/share/prometheus/consoles'
- '--web.console.libraries=/usr/share/prometheus/console_librari

ports:

- # Hardening: Expose only to localhost. Use Nginx/Ingress for ext*
- "127.0.0.1:9090:9090"

networks:

- monitoring-net

healthcheck:

test: ["CMD", "wget", "-q", "--tries=1", "-O-", "http://localhos
interval: 30s
timeout: 10s
retries: 3

logging:

driver: "json-file"
options:
 max-size: "10m"
 max-file: "3"

node-exporter:

image: prom/node-exporter:v1.8.2

container_name: node-exporter-prod

restart: unless-stopped

volumes:

- /proc:/host/proc:ro
- /sys:/host/sys:ro
- /:/rootfs:ro

```

command:
  - '--path.procfs=/host/proc'
  - '--path.sysfs=/host/sys'
  - '--path.rootfs=/rootfs'
  # Production Hardening: Strip out irrelevant mount spaces to opt
  - '--collector.filesystem.mount-points-exclude=^/(sys|proc|dev|h
ports:
  - "127.0.0.1:9100:9100"
networks:
  - monitoring-net
logging:
  driver: "json-file"
  options:
    max-size: "10m"
    max-file: "3"

networks:
  monitoring-net:
    driver: bridge

volumes:
  prometheus-data:
    driver: local

```

Use code with caution.

Step 3: Architecting Scrape Targets (`prometheus/prometheus.yml`)

Save the primary execution script under `prometheus/prometheus.yml` . It defines global scraping standards, populates environment tags for metrics pipelines, and drops port details via Regex labels.

yaml

```

global:
  scrape_interval: 15s      # Standard enterprise interval for metric
  evaluation_interval: 15s  # How frequently alert conditions are comp
  scrape_timeout: 10s
  external_labels:
    environment: 'production'
    region: 'us-west-2'
    cluster: 'docker-prod-01'

rule_files:
  - "alert.rules.yml"

```

```

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['prometheus:9090']

  - job_name: 'node-exporter'
    static_configs:
      - targets: ['node-exporter:9100']
    relabel_configs:
      # Production Label Cleaning: Removes ports from titles for clean
      - source_labels: [__address__]
        target_label: instance
        regex: '([^:]+)(:\d+)?'
        replacement: '${1}'

```

Use code with caution.

Step 4: Structuring Infrastructure Thresholds (`prometheus/alert.rules.yml`)

Create the alert engine configuration file under `prometheus/alert.rules.yml` .
This maps the mathematical parameters triggering system telemetry notifications.

yaml

```

groups:
  - name: infrastructure_alerts
    rules:
      - alert: HostHighCpuLoad
        expr: 100 - (avg by(instance) (rate(node_cpu_seconds_total{mod
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "High CPU load on host {{ $labels.instance }}"
          description: "CPU load is {{ $value | printf \"%.2f\" }}% fo

      - alert: HostOutOfMemory
        expr: (node_memory_MemAvailable_bytes / node_memory_MemTotal_b
        for: 2m
        labels:
          severity: critical
        annotations:

```

```
summary: "Host out of memory on {{ $labels.instance }}"
description: "Available memory is below 10% (current: {{ $va

- alert: HostDiskSpaceRunningLow
  expr: (node_filesystem_avail_bytes{mountpoint="/" } / node_file
  for: 5m
  labels:
    severity: warning
  annotations:
    summary: "Low disk space on {{ $labels.instance }}"
    description: "Free disk space on root filesystem is below 15

- alert: InstanceDown
  expr: up == 0
  for: 1m
  labels:
    severity: critical
  annotations:
    summary: "Instance {{ $labels.instance }} down"
    description: "{{ $labels.instance }} of job {{ $labels.job }}
```

Use code with caution.

Step 5: Stack Execution and Lifecycle Administration

Deploy the infrastructure with the detach argument:

```
bash
```

```
docker compose up -d
```

Use code with caution.

Verify the engine health:

```
bash
```

```
docker compose ps
```

Use code with caution.

Because production mode is started with the `--web.enable-lifecycle` flag configuration, updates can be dynamically applied **without restarting the stack** by making an internal POST request to the runtime reload engine:

```
bash
```

```
curl -X POST http://127.0.0.0
```

Use code with caution.

Would you like to build an internal **Nginx Reverse Proxy configuration with Basic Authentication** to securely expose this stack outside of localhost, or should we write a configuration layout to integrate an **Alertmanager container** for dispatching notifications directly to **Slack or PagerDuty**?



how to test if both the deployed containers are working or not.

To verify that your production monitoring stack is fully operational, you need to test three distinct layers: process execution, network connectivity, and telemetry collection .

Here is how to run these diagnostic tests step-by-step from your terminal.

Test 1: Validate Container Runtime and Health States

Because we configured native `healthcheck` keys in the compose file, Docker actively monitors the inner states of the processes rather than just checking if the container is "running".

Run this command in your project directory:

```
bash
```

```
docker compose ps
```

Use code with caution.

Expected Output:

text

NAME	IMAGE	COMMAND
node-exporter-prod	prom/node-exporter:v1.8.2	"/bin/node_exporter
prometheus-prod	prom/prometheus:v2.54.0	"/bin/prometheus --

Use code with caution.

- Look closely at the **STATUS** column. `prometheus-prod` must explicitly display `running (healthy)`.
- If it displays `unhealthy` or `exited`, run `docker compose logs prometheus` to review the configuration parser errors.

Test 2: Verify Node Exporter Metric Broadcasts

Node Exporter reads host kernel data and translates it into a plaintext format that Prometheus can parse. You can verify that it is capturing infrastructure data by scraping its port locally.

Run this curl command from the host machine:

bash

```
curl -s http://127.0.0 | head -n 15
```

Use code with caution.

Expected Output:

text

```
# HELP go_gc_duration_seconds A summary of the wall-time pause total d
# TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 0
go_gc_duration_seconds{quantile="0.25"} 0
...
```

```
# HELP node_cpu_seconds_total Seconds the CPUs spent in each mode.
# TYPE node_cpu_seconds_total counter
node_cpu_seconds_total{cpu="0",mode="idle"} 12845.32
```

Use code with caution.

- If you see structured lines starting with `# HELP` and `# TYPE`, followed by raw metrics like `node_cpu_seconds_total`, **Node Exporter is successfully gathering system metrics.**

Test 3: Query the Prometheus Scrape Target Engine

Next, you need to ensure Prometheus isn't just running, but is actually pulling data from Node Exporter over the isolated internal network bridge. You can query the internal Prometheus API directly from your terminal.

Run this command to check the status of all active scraping pipelines:

```
bash
```

```
curl -s http://127.0.0.1 | json_pp 2>/dev/null || curl -s http://127.0.0.1
```

Use code with caution.

Expected Output (Look for the `health` fields):

```
json
```

```
{
  "status": "success",
  "data": {
    "activeTargets": [
      {
        "discoveredLabels": { "__address__": "node-exporter:9100" },
        "labels": { "instance": "node-exporter", "job": "node-exporter",
        "scrapeUrl": "http://node-exporter:9100/metrics",
        "health": "up"
      },
      {
        "discoveredLabels": { "__address__": "prometheus:9090" },
        "labels": { "instance": "prometheus", "job": "prometheus" },
```

```
    "scrapeUrl": "http://prometheus:9090/metrics",  
    "health": "up"  
  }  
]  
}  
}
```

Use code with caution.

- **The Ultimate Check:** Both entries inside the array must return `"health": "up"`.
- If a target shows `"health": "down"`, it means Prometheus cannot route traffic to that container across the virtual network bridge.

Test 4: Access the Native Web Interface

Since port `9090` is bound strictly to `127.0.0.1` for production security, it will reject direct external web traffic. To securely view the graph interface on your local workstation, use an **SSH Tunnel**:

1. Run this terminal command on your local laptop to tunnel the port securely:

```
bash
```

```
ssh -L 9090:127.0.0.1:9090 user@your-server-ip
```

Use code with caution.

2. Open your local browser and navigate to: `http://localhost:9090/targets`
3. You will see the Prometheus web portal displaying green `UP` badges next to both endpoints.

Now that your stack is verified and running, would you like to ↪ **simulate a high CPU load** to test if the alerting rules we configured actually trigger, or should we move on to configuring a ↪ **Grafana container** to build a visual dashboard?



great. Now, tell me sample five promQL query that i can run and get some very useful information based on above ...

Here are five production-ready PromQL queries tailored exactly to the **Node Exporter** and **Prometheus** components we just deployed.

You can execute these directly by typing them into the expression bar in the Prometheus Expression Browser (`http://localhost:9090`).

1. Actual System CPU Utilization Percentage

The raw metric `node_cpu_seconds_total` is a counter that ticks upward indefinitely for each CPU core and execution mode. To find out how busy your system actually is right now, we calculate the per-second rate of the **idle** mode over the last 5 minutes, average it across all cores, and subtract it from 100%.

promql

```
100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m]
```

Use code with caution.

- **Why it's useful:** Gives you a single, accurate efficiency figure (0% to 100%) representing the overall processing load of your server, matching what you would see in native utilities like `top` or `htop`.

2. Actual Available Memory (RAM) Percentage

Many administrators mistakenly look at free memory (`node_memory_MemFree_bytes`), which causes false alarms because Linux aggressively caches files in inactive RAM. This query accurately compares true *available* memory (including reclaimable caches) against total hardware capacity.

promql

```
(node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes) * 100
```

Use code with caution.

- **Why it's useful:** Tells you exactly how much memory headroom your machine has left before it runs the risk of hitting Out-Of-Memory (OOM) triggers and killing containers.

3. Disk Space Usage Percentage (Root Filesystem)

This query tracks storage consumption across your root mount point. It subtracts available bytes from the total size to find out how much space is occupied, then converts it into a clean percentage.

promql

```
((node_filesystem_size_bytes{mountpoint="/"} - node_filesystem_avail_b
```

Use code with caution.

- **Why it's useful:** Essential for proactive storage planning. Docker containers constantly write runtime logs and build structural layers that can quietly saturate system storage.

4. Disk Storage Saturation Prediction (Linear Regression)

Instead of waiting for disk space to completely fill up, this advanced query uses the `deriv` function to analyze storage write patterns over the past 24 hours. It then uses linear regression to calculate how many **hours** remain until the disk runs out of space if current consumption behavior continues.

promql

```
node_filesystem_free_bytes{mountpoint="/"} / -deriv(node_filesystem_fr
```

Use code with caution.

- **Why it's useful: Invaluable for DevOps on-call engineers.** If a broken container starts aggressively writing debug logs, this query will tell you exactly how many hours you have to fix the issue before the drive completely locks up. *(Note: If the result is negative, it means your disk space is currently clearing or stable).*

5. Prometheus Scraping Efficiency (Samples Scraped Per Second)

This query shifts focus inward to monitor the health of your monitoring infrastructure itself. It calculates the total volume of raw time-series data strings ingested by your scraper every second.

```
promql
```

```
sum by (job) (rate(prometheus_tsdb_head_samples_appended_total[5m]))
```

Use code with caution.

- **Why it's useful:** Helps you keep an eye on your monitoring overhead. If you notice this number spiking unexpectedly into the thousands, it indicates that a newly deployed target is introducing metric bloat and overloading your TSDB engine.

Pro-Tip: Adding These to Your Target Configurations

You can test these expressions interactively on the **Graph** tab in the Prometheus UI to see real-time trends over time.

Would you like to ↪**inject a simulation payload** to watch these numbers spike dynamically, or should we connect a ↪**Grafana dashboard** to turn these raw mathematical formulas into visual monitoring charts?



[inject a simulation payload](#)

To test your Prometheus implementation and verify that your `alert.rules.yml` thresholds actually work, we will safely inject artificial stress into the system.

Since your current containers are bound to `127.0.0.1` for production security, you must execute these simulation commands **directly on the host machine** where Docker is running.

Payload 1: Simulate High CPU Load (Triggers `HostHighCpuLoad`)

Our configuration triggers a `warning` alert if CPU utilization stays above **85% for 5 minutes**. We will use `stress-ng` (or a native Linux container) to spike the processor cores.

Run this command on your host terminal to spin up a temporary, resource-restricted container that will stress all available CPU cores:

```
bash
```

```
docker run --rm -it --name cpu-stresser alpine sh -c "apk add --no-cac
```

Use code with caution.

- **How to verify:** Open Prometheus (`http://localhost:9090`) and execute your CPU PromQL query:

```
100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
```

You will see the line jump straight up to ~95%.

- **How to stop it:** Press `Ctrl + C` in that terminal window, or run `docker stop cpu-stresser` .

Payload 2: Simulate High Memory Consumption (Triggers

`HostOutOfMemory`)

Our rule triggers a `critical` alert if available memory drops **below 10% for 2 minutes**. We can use a secure `stress-ng` memory allocation loop.

First, check your total available memory so you don't crash your server. If you have 4GB of RAM, we want to temporarily lock up about 3.8GB of it. Run this command (adjust the `3.8G` parameter safely based on your system size):

```
bash
```

```
docker run --rm -it --name mem-stresser alpine sh -c "apk add --no-cac
```

Use code with caution.

- **How to verify:** Run your memory PromQL query:

```
(node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes) * 100
```

The percentage will plummet below your 10% critical threshold.

- **How to stop it:** It will auto-terminate after 10 minutes, or you can force stop it immediately with `docker stop mem-stresser`.

Payload 3: Simulate a Node Outage (Triggers `InstanceDown`)

Our rule triggers a `critical` alert if any scraped instance goes down **for more than 1 minute** (`up == 0`). The cleanest way to test this infrastructure lifecycle path is to intentionally stop the exporter container.

```
bash
```

```
docker compose stop node-exporter
```

Use code with caution.

- **How to verify:** Run the PromQL query: `up`. The metric value for `node-exporter` will instantly flip from `1` (healthy) to `0` (disconnected).
- **How to recover:** Bring the node exporter back online by running:

```
bash
```

```
docker compose start node-exporter
```

Use code with caution.

Track the Alert Lifecycle in the Prometheus UI

As you execute these payloads, you can watch Prometheus transition through its internal state machine in real time. Open your browser and navigate to

```
http://localhost:9090/alerts :
```

1. `Inactive` (**Green**): The system state is completely normal.

2. **Pending (Amber)**: The metric has crossed the threshold (e.g., CPU is at 95%), but the duration condition (`for: 5m`) has not met the threshold yet. This prevents transient metric spikes from waking up engineering teams.
3. **Firing (Red)**: The metric has breached the threshold for longer than the defined duration. The alert engine is now actively screaming for resolution.

Now that you know how to validate and fire alerts, would you like to configure ↪ **Alertmanager** to route these firing alerts directly to ↪ **Slack/PagerDuty**, or should we introduce ↪ **Grafana** to create a visual dashboard for these spikes?



Alertmanager

Let's layer **Alertmanager** into our production stack. In an enterprise topology, Prometheus is strictly responsible for evaluating alerting rules, while Alertmanager acts as a decoupled routing engine that deduplicates, groups, and dispatches those alerts to messaging platforms like Slack or PagerDuty.

Here is how to integrate Alertmanager into your existing Docker Compose layout step-by-step.

Step 1: Update the Directory Architecture

Run this command on your host machine to create a dedicated configuration home for Alertmanager:

```
bash
```

```
mkdir -p prometheus/alertmanager
```

Use code with caution.

Step 2: Create the Alertmanager Configuration File

Create and save this file as `prometheus/alertmanager/alertmanager.yml`. It aggregates alerts by rule name over a 30-second window to prevent alert storms

when a major network or rack failure occurs.

yaml

```
global:
  resolve_timeout: 5m

route:
  group_by: ['alertname', 'cluster', 'environment']
  group_wait: 30s      # How long to wait to buffer initial alerts of
  group_interval: 5m   # How long to wait before sending a bundle of n
  repeat_interval: 12h # How long to wait before re-sending a duplicat
  receiver: 'slack-notifications'

receivers:
- name: 'slack-notifications'
  slack_configs:
  - api_url: 'https://slack.com' # Replace with real Slack Webhook
    channel: '#ops-alerts'
    send_resolved: true
    title: '[[{ .Status | toUpper }]] [{ .CommonLabels.alertname }] -
    text: >-
      {{ range .Alerts }}
        *Alert Details:* [{ .Annotations.description }]
        *Severity:* `[{ .Labels.severity }]`
        *Instance:* [{ .Labels.instance }]
      {{ end }}
```

Use code with caution.

Step 3: Inject Alertmanager into `docker-compose.yml`

Modify your existing root `docker-compose.yml` file to include the alertmanager service container, connect it to the network bridge, and add a data storage volume.

yaml

```
version: '3.8'

services:
  prometheus:
```

```
image: prom/prometheus:v2.54.0
container_name: prometheus-prod
restart: unless-stopped
volumes:
  - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
  - ./prometheus/alert.rules.yml:/etc/prometheus/alert.rules.yml:ro
  - prometheus-data:/prometheus
command:
  - '--config.file=/etc/prometheus/prometheus.yml'
  - '--storage.tsdb.path=/prometheus'
  - '--storage.tsdb.retention.time=30d'
  - '--storage.tsdb.retention.size=50GB'
  - '--web.enable-lifecycle'
ports:
  - "127.0.0.1:9090:9090"
networks:
  - monitoring-net
depends_on:
  - alertmanager
```

alertmanager:

```
image: prom/alertmanager:v0.27.0
container_name: alertmanager-prod
restart: unless-stopped
volumes:
  - ./prometheus/alertmanager/alertmanager.yml:/etc/alertmanager/alertmanager.yml:ro
  - alertmanager-data:/alertmanager
command:
  - '--config.file=/etc/alertmanager/alertmanager.yml'
  - '--storage.path=/alertmanager'
ports:
  - "127.0.0.1:9093:9093"
networks:
  - monitoring-net
healthcheck:
  test: ["CMD", "wget", "-q", "--tries=1", "-O-", "http://localhost:9093/health"]
  interval: 30s
  timeout: 10s
  retries: 3
```

node-exporter:

```
image: prom/node-exporter:v1.8.2
container_name: node-exporter-prod
restart: unless-stopped
volumes:
  - /proc:/host/proc:ro
  - /sys:/host/sys:ro
```

```

- /:/rootfs:ro
command:
- '--path.procfs=/host/proc'
- '--path.sysfs=/host/sys'
- '--path.rootfs=/rootfs'
- '--collector.filesystem.mount-points-exclude=^/(sys|proc|dev|h
ports:
- "127.0.0.1:9100:9100"
networks:
- monitoring-net

networks:
  monitoring-net:
    driver: bridge

volumes:
  prometheus-data:
    driver: local
  alertmanager-data:
    driver: local

```

Use code with caution.

Step 4: Map Prometheus to Alertmanager

(`prometheus/prometheus.yml`)

Open your existing `prometheus/prometheus.yml` and add the `alerting` configuration root right under the global block. This tells Prometheus where to dispatch firing alert payloads.

yaml

```

global:
  scrape_interval: 15s
  evaluation_interval: 15s

# Connect Prometheus to the Local Alertmanager container
alerting:
  alertmanagers:
    - static_configs:
      - targets:
        - 'alertmanager:9093'

```

```
rule_files:
  - "alert.rules.yml"

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['prometheus:9090']

  - job_name: 'node-exporter'
    static_configs:
      - targets: ['node-exporter:9100']
```

Use code with caution.

Step 5: Start the Architecture Update

Deploy the changes to create the alertmanager container and update the running Prometheus instance:

```
bash
```

```
docker compose up -d
```

Use code with caution.

Verify that all three components are functional and running by typing:

```
bash
```

```
docker compose ps
```

Use code with caution.

To see the native alertmanager engine dash securely from your workspace laptop, create a network tunnel:

```
bash
```

```
ssh -L 9093:127.0.0.1:9093 user@your-server-ip
```

Use code with caution.

Then navigate your laptop browser to `http://localhost:9093`. When you inject a simulation payload (like stopping node-exporter via `docker compose stop node-exporter`), you will now watch the alert land in Alertmanager and route directly out via your Slack Webhook URL.

Would you like to extend this routing strategy by implementing ↪ **advanced Alertmanager routing trees** (e.g., routing `critical` items to ↪ **PagerDuty/Opsgenie** and `warning` items to ↪ **Slack**), or should we complete the stack by attaching a ↪ **Grafana server** container?



Grafana server

Let's complete your production observability ecosystem by layers-in **Grafana**. In a production setup, Grafana serves as the unified visualization and dashboarding layer, querying data directly from Prometheus without adding compute overhead to your application workloads.

Step 1: Initialize Provisioning Directories

To prevent engineers from losing their dashboard configurations if a container restarts, we will adopt a **GitOps provisioning approach**. This allows us to hardcode the Prometheus data source using configuration files.

Run these terminal commands on your host system:

```
bash
```

```
mkdir -p prometheus/grafana/provisioning/datasources
```

Use code with caution.

Step 2: Auto-Provision the Prometheus Data Source

Create and save this file as

`prometheus/grafana/provisioning/datasources/prometheus.yml` . When Grafana spins up, it will automatically connect to your Prometheus backend over the internal Docker network.

yaml

```
apiVersion: 1

datasources:
- name: Prometheus
  type: prometheus
  access: proxy
  url: http://prometheus:9090
  isDefault: true
  editable: false # Production Hardening: Locks the UI settings down
  jsonData:
    httpMethod: POST
    timeInterval: 15s
```

Use code with caution.

Step 3: Append Grafana to the Orchestrator (`docker-compose.yml`)

Open your root `docker-compose.yml` file and add the `grafana` service configuration block. We will explicitly lock down container permissions using an unprivileged runtime user ID (`472` is Grafana's official user ID).

yaml

```
version: '3.8'

services:
  prometheus:
    image: prom/prometheus:v2.54.0
    container_name: prometheus-prod
    restart: unless-stopped
    volumes:
```

- ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
- ./prometheus/alert.rules.yml:/etc/prometheus/alert.rules.yml:r
- prometheus-data:/prometheus

command:

- '--config.file=/etc/prometheus/prometheus.yml'
- '--storage.tsdb.path=/prometheus'
- '--storage.tsdb.retention.time=30d'
- '--storage.tsdb.retention.size=50GB'
- '--web.enable-lifecycle'

ports:

- "127.0.0.1:9090:9090"

networks:

- monitoring-net

depends_on:

- alertmanager

alertmanager:

image: prom/alertmanager:v0.27.0

container_name: alertmanager-prod

restart: unless-stopped

volumes:

- ./prometheus/alertmanager/alertmanager.yml:/etc/alertmanager/a
- alertmanager-data:/alertmanager

command:

- '--config.file=/etc/alertmanager/alertmanager.yml'
- '--storage.path=/alertmanager'

ports:

- "127.0.0.1:9093:9093"

networks:

- monitoring-net

grafana:

image: grafana/grafana:11.1.0

container_name: grafana-prod

restart: unless-stopped

user: "472" # Production Hardening: Explicitly forces unprivileged

ports:

- # Hardening: Exposed to Localhost. Use an Nginx proxy or VPN end
- "127.0.0.1:3000:3000"

volumes:

- ./prometheus/grafana/provisioning:/etc/grafana/provisioning:ro
- grafana-data:/var/lib/grafana

environment:

- GF_SECURITY_ADMIN_USER=admin
- GF_SECURITY_ADMIN_PASSWORD=SuperSecretProductionPassword123! #
- GF_USERS_ALLOW_SIGN_UP=false # Production Hardening: Disables
- GF_ANALYTICS_REPORTING_ENABLED=false

```
networks:
  - monitoring-net
depends_on:
  - prometheus
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:3000/api/health"]
  interval: 30s
  timeout: 10s
  retries: 3
```

```
node-exporter:
  image: prom/node-exporter:v1.8.2
  container_name: node-exporter-prod
  restart: unless-stopped
  volumes:
    - /proc:/host/proc:ro
    - /sys:/host/sys:ro
    - /:/rootfs:ro
  command:
    - '--path.procfs=/host/proc'
    - '--path.sysfs=/host/sys'
    - '--path.rootfs=/rootfs'
    - '--collector.filesystem.mount-points-exclude=^/(sys|proc|dev|h
ports:
  - "127.0.0.1:9100:9100"
networks:
  - monitoring-net
```

```
networks:
  monitoring-net:
    driver: bridge
```

```
volumes:
  prometheus-data:
    driver: local
  alertmanager-data:
    driver: local
  grafana-data:
    driver: local
```

Use code with caution.

Step 4: Apply Storage Permissions & Deploy

Before launching, make sure the local directory engine permissions allow the nested container processes to access them cleanly:

```
bash
```

```
sudo chown -R 472:472 prometheus/grafana/
```

Use code with caution.

Now, pull the images and spin up the complete, integrated metrics ecosystem:

```
bash
```

```
docker compose up -d
```

Use code with caution.

Verify that all stack services are running healthy:

```
bash
```

```
docker compose ps
```

Use code with caution.

Step 5: Secure Local Verification & Dashboard Setup

1. Establish a secure **SSH Tunnel** from your local engineering laptop to access the dashboard port:

```
bash
```

```
ssh -L 3000:127.0.0.1:3000 user@your-server-ip
```

Use code with caution.

2. Navigate your local web browser to: `http://localhost:3000`

3. Log in using the credentials defined in your compose environment file (`admin` / `SuperSecretProductionPassword123!`).
4. **Import an Enterprise-Grade Dashboard:** You do not need to build node dashboards from scratch.
 1. In the top-right navigation menu, click the `+` icon -> **Import** .
 2. Enter the official community ID for Node Exporter: `1860` (or `10175` for a refined look).
 3. Click **Load**, select the auto-provisioned **Prometheus** datasource dropdown, and click **Import**.

You will instantly see enterprise panels rendering high-fidelity graphs detailing your CPU, Memory, Disk, and Network metrics using the PromQL configurations we created!

Would you like to auto-provision this specific **Node Exporter dashboard via code** so that Grafana boots up with the visual dashboard already loaded out-of-the-box, or should we talk about configuring **Logs Integration** using Loki?